TP n°4

Introduction au
Procedural Language (PL/SQL) Oracle

Partie I

Nom : Mouaden

Prénom : Alae

Module : Bases de données avancées

0 Table des matières

1 Exercice 1 – Blocs anonymes de base **2**
1.1 Somme de deux entiers 2
1.2 Table de multiplication 2
1.3 Fonction récursive : x^n 3
1.4 Factorielle d'un nombre (stockage en table) 3
1.5 Factorielles des 20 premiers entiers 4

2 Exercice 2 – Gestion des employés **5**
2.1 Création et peuplement de la table `emp` 5
2.2 Insertion d'un employé via %ROWTYPE 5
2.3 Suppression avec `SQL%ROWCOUNT` 6
2.4 Somme des salaires – curseur explicite (LOOP) 6
2.5 Salaire moyen – curseur explicite (LOOP) 7
2.6 Réécriture avec la boucle `FOR IN` 8
2.7 Curseur paramétré 9

3 Exercice 3 – Package de gestion des clients **10**
3.1 Spécification du package 10
3.2 Corps du package 10
3.3 Tests du package 11

1 Exercice 1 – Blocs anonymes de base

Rappel : pour activer l'affichage sous SQL Developer, aller dans *Affichage* et cliquer sur *Sortie SGBD*, puis ajouter `SET SERVEROUTPUT ON;` en début de programme.

1.1 Somme de deux entiers

Énoncé

Écrire une procédure anonyme PL/SQL qui demande à l'utilisateur de saisir deux entiers et affiche leur somme.

```
SET SERVEROUTPUT ON;

DECLARE
    -- Saisie interactive via les variables de substitution SQL*Plus
    v_num1  NUMBER := &premier_entier;
    v_num2  NUMBER := &deuxieme_entier;
    v_somme NUMBER;
BEGIN
    v_somme := v_num1 + v_num2;
    DBMS_OUTPUT.PUT_LINE(
        'Somme de ' || v_num1 || ' + ' || v_num2 || ' = ' || v_somme
    );
END;
/
```

Listing 1 – Somme de deux entiers saisis

1.2 Table de multiplication

Énoncé

Écrire une procédure anonyme PL/SQL qui demande un nombre et affiche sa table de multiplication.

```
SET SERVEROUTPUT ON;

DECLARE
    v_nombre NUMBER := &nombre;   -- Nombre dont on veut la table
    i        NUMBER := 1;         -- Compteur de boucle
BEGIN
    DBMS_OUTPUT.PUT_LINE('=== Table de multiplication de '
                         || v_nombre || ' ===');

    -- Boucle LOOP avec sortie explicite apres 10 iterations
    LOOP
        DBMS_OUTPUT.PUT_LINE(
            v_nombre || ' x ' || i || ' = ' || (v_nombre * i)
        );
        i := i + 1;
        EXIT WHEN i > 10;
    END LOOP;
END;
/
```

Listing 2 – Table de multiplication d'un nombre

1.3 Fonction récursive : x^n

Énoncé

Écrire une fonction récursive qui retourne x^n (x et n entiers positifs).

```oracle
CREATE OR REPLACE FUNCTION puissance(x IN NUMBER, n IN NUMBER)
RETURN NUMBER
IS
BEGIN
    -- Cas de base : tout nombre a la puissance 0 vaut 1
    IF n = 0 THEN
        RETURN 1;
    ELSE
        -- Appel recursif : x^n = x * x^(n-1)
        RETURN x * puissance(x, n - 1);
    END IF;
END;
/

-- Test de la fonction
BEGIN
    DBMS_OUTPUT.PUT_LINE('2^10 = ' || puissance(2, 10));
    DBMS_OUTPUT.PUT_LINE('3^4 = ' || puissance(3, 4));
END;
/
```

Listing 3 – Fonction rÃ©cursive de puissance

1.4 Factorielle d'un nombre (stockage en table)

Énoncé

Écrire une procédure anonyme qui calcule la factorielle d'un nombre strictement positif saisi par l'utilisateur et stocke le résultat dans une table `resultatFactoriel`.

```oracle
-- Creation de la table (a executer une seule fois)
CREATE TABLE resultatFactoriel (
    nombre      NUMBER(10),
    factorielle NUMBER(38)
);

SET SERVEROUTPUT ON;

DECLARE
    v_nombre      NUMBER := &nombre_factoriel;
    v_factorielle NUMBER := 1;
    i             NUMBER := 1;
BEGIN
    -- Verification : le nombre doit etre strictement positif
    IF v_nombre <= 0 THEN
        DBMS_OUTPUT.PUT_LINE(
```

```
            'Erreur : le nombre doit etre strictement positif.'
        );
    ELSE
        -- Calcul iteratif : n! = 1 * 2 * 3 * ... * n
        LOOP
            v_factorielle := v_factorielle * i;
            i := i + 1;
            EXIT WHEN i > v_nombre;
        END LOOP;

        -- Stockage du resultat en base
        INSERT INTO resultatFactoriel (nombre, factorielle)
        VALUES (v_nombre, v_factorielle);

        DBMS_OUTPUT.PUT_LINE(v_nombre || '! = ' || v_factorielle);
        COMMIT;
    END IF;
END;
/
```

Listing 4 – Factorielle avec stockage dans une table

1.5 Factorielles des 20 premiers entiers

Énoncé

Modifier le programme précédent pour calculer et stocker dans une table `resultatsFactoriels` les factorielles des 20 premiers entiers.

```
-- Creation de la table de stockage
CREATE TABLE resultatsFactoriels (
    nombre      NUMBER(10),
    factorielle NUMBER(38)
);

SET SERVEROUTPUT ON;

DECLARE
    -- On accumule la factorielle courante pour eviter de tout
       recalculer
    -- Propriete : n! = (n-1)! * n
    v_factorielle NUMBER := 1;
BEGIN
    FOR n IN 1..20 LOOP
        v_factorielle := v_factorielle * n;

        INSERT INTO resultatsFactoriels (nombre, factorielle)
        VALUES (n, v_factorielle);

        DBMS_OUTPUT.PUT_LINE(n || '! = ' || v_factorielle);
    END LOOP;

    COMMIT;
END;
/
```

Listing 5 – Factorielles de 1 Ã 20 avec boucle FOR IN

2 Exercice 2 – Gestion des employés

2.1 Création et peuplement de la table emp

```sql
CREATE TABLE emp (
    matr     NUMBER(10)   NOT NULL,
    nom      VARCHAR2(50) NOT NULL,
    sal      NUMBER(7,2),
    adresse  VARCHAR2(96),
    dep      NUMBER(10)   NOT NULL,
    CONSTRAINT emp_pk PRIMARY KEY (matr)
);

-- Donnees de test
INSERT INTO emp VALUES (1, 'Alice', 3000, '10 rue de Paris', 75000);
INSERT INTO emp VALUES (2, 'Bob', 2800, '5 avenue Victor Hugo',
    92000);
INSERT INTO emp VALUES (3, 'Claire', 3200, '20 boulevard Voltaire',
    75000);
INSERT INTO emp VALUES (5, 'David', 2600, '8 rue Nationale', 10);
COMMIT;
```

Listing 6 – CrÃ©ation de la table emp et insertion de donnÃ©es

2.2 Insertion d'un employé via %ROWTYPE

Énoncé

Écrire un bloc anonyme qui insère un nouvel employé (matr=4, nom=Youcef, sal=2500, adresse=avenue de la République, dep=92002).

```sql
SET SERVEROUTPUT ON;

DECLARE
    -- %ROWTYPE cree une variable ayant exactement la structure de la
        table emp
    v_employe emp%ROWTYPE;
BEGIN
    v_employe.matr    := 4;
    v_employe.nom     := 'Youcef';
    v_employe.sal     := 2500;
    v_employe.adresse := 'avenue de la Republique';
    v_employe.dep     := 92002;

    -- Insertion de l'enregistrement complet en une seule instruction
    INSERT INTO emp VALUES v_employe;

    DBMS_OUTPUT.PUT_LINE(
        'Employe ' || v_employe.nom || ' insere avec succes.'
    );
    COMMIT;
END;
/
```

Listing 7 – Insertion d'un employÃ© avec %ROWTYPE

2.3 Suppression avec SQL%ROWCOUNT

Énoncé

Écrire un bloc anonyme qui supprime tous les employés d'un département et affiche le nombre de n-uplets supprimés via `SQL%ROWCOUNT`.

```
SET SERVEROUTPUT ON;

DECLARE
    v_nb_lignes NUMBER;
    v_dep       NUMBER := 10;
BEGIN
    DELETE FROM emp WHERE dep = v_dep;

    -- SQL%ROWCOUNT : nombre de lignes affectees par le dernier ordre
        DML
    v_nb_lignes := SQL%ROWCOUNT;

    DBMS_OUTPUT.PUT_LINE(
        'Nombre d employes supprimes du dep. '
        || v_dep || ' : ' || v_nb_lignes
    );
    COMMIT;
END;
/
```

Listing 8 – Suppression avec comptage via SQL%ROWCOUNT

2.4 Somme des salaires – curseur explicite (LOOP)

Énoncé

Écrire un bloc anonyme affichant la somme des salaires de tous les employés, en utilisant un curseur explicite et la boucle `LOOP...END LOOP`. Résultat équivalent à `SELECT SUM(sal) FROM emp;`.

```
SET SERVEROUTPUT ON;

DECLARE
    v_salaire emp.sal%TYPE;
    v_total   emp.sal%TYPE := 0;

    CURSOR c_salaires IS
        SELECT sal FROM emp;
BEGIN
    OPEN c_salaires;

    LOOP
        FETCH c_salaires INTO v_salaire;
        EXIT WHEN c_salaires%NOTFOUND;

        IF v_salaire IS NOT NULL THEN
            v_total := v_total + v_salaire;
        END IF;
    END LOOP;

```

```
    CLOSE c_salaires;

    DBMS_OUTPUT.PUT_LINE('Somme des salaires : ' || v_total);
END;
/
```

Listing 9 – Somme des salaires avec curseur explicite

2.5 Salaire moyen – curseur explicite (LOOP)

Énoncé

Modifier la procédure précédente pour calculer le salaire moyen. Résultat équivalent à `SELECT AVG(sal) FROM emp;`.

```
SET SERVEROUTPUT ON;

DECLARE
    v_salaire emp.sal%TYPE;
    v_total   emp.sal%TYPE := 0;
    v_nb_emp  NUMBER       := 0;
    v_moyenne NUMBER;

    CURSOR c_salaires IS
        SELECT sal FROM emp;
BEGIN
    OPEN c_salaires;

    LOOP
        FETCH c_salaires INTO v_salaire;
        EXIT WHEN c_salaires%NOTFOUND;

        IF v_salaire IS NOT NULL THEN
            v_total  := v_total + v_salaire;
            v_nb_emp := v_nb_emp + 1;
        END IF;
    END LOOP;

    CLOSE c_salaires;

    IF v_nb_emp > 0 THEN
        v_moyenne := v_total / v_nb_emp;
        DBMS_OUTPUT.PUT_LINE('Salaire moyen : ' || ROUND(v_moyenne,
            2));
    ELSE
        DBMS_OUTPUT.PUT_LINE('Aucun salaire renseigne.');
    END IF;
END;
/
```

Listing 10 – Salaire moyen avec curseur explicite

2.6 Réécriture avec la boucle FOR IN

Énoncé

Modifier les deux procédures précédentes en utilisant la boucle `FOR...IN`. La boucle `FOR IN` gère automatiquement `OPEN`, `FETCH` et `CLOSE`.

```oracle
SET SERVEROUTPUT ON;

DECLARE
    v_total emp.sal%TYPE := 0;
    CURSOR c_salaires IS SELECT sal FROM emp;
BEGIN
    FOR v_rec IN c_salaires LOOP
        IF v_rec.sal IS NOT NULL THEN
            v_total := v_total + v_rec.sal;
        END IF;
    END LOOP;

    DBMS_OUTPUT.PUT_LINE('[FOR IN] Somme des salaires : ' || v_total);
END;
/
```

Listing 11 – Somme des salaires avec FOR IN

```oracle
SET SERVEROUTPUT ON;

DECLARE
    v_total  emp.sal%TYPE := 0;
    v_nb_emp NUMBER       := 0;
    CURSOR c_salaires IS SELECT sal FROM emp;
BEGIN
    FOR v_rec IN c_salaires LOOP
        IF v_rec.sal IS NOT NULL THEN
            v_total  := v_total + v_rec.sal;
            v_nb_emp := v_nb_emp + 1;
        END IF;
    END LOOP;

    IF v_nb_emp > 0 THEN
        DBMS_OUTPUT.PUT_LINE(
            '[FOR IN] Salaire moyen : ' || ROUND(v_total / v_nb_emp, 2)
        );
    ELSE
        DBMS_OUTPUT.PUT_LINE('[FOR IN] Aucun salaire renseigne.');
    END IF;
END;
/
```

Listing 12 – Salaire moyen avec FOR IN

2.7 Curseur paramétré

Énoncé

Écrire une procédure anonyme qui affiche les noms des employés des départements 92000 et 75000 en utilisant un curseur paramétré.

```
SET SERVEROUTPUT ON;

DECLARE
    CURSOR c(p_dep emp.dep%TYPE) IS
        SELECT dep, nom
        FROM emp
        WHERE dep = p_dep;
BEGIN
    DBMS_OUTPUT.PUT_LINE('--- Employes du departement 92000 ---');
    FOR v_employe IN c(92000) LOOP
        DBMS_OUTPUT.PUT_LINE('  ' || v_employe.nom);
    END LOOP;

    DBMS_OUTPUT.PUT_LINE('--- Employes du departement 75000 ---');
    FOR v_employe IN c(75000) LOOP
        DBMS_OUTPUT.PUT_LINE('  ' || v_employe.nom);
    END LOOP;
END;
/
```

Listing 13 – Curseur paramÃ©trÃ© pour deux dÃ©partements

3 Exercice 3 – Package de gestion des clients

Énoncé

Implémenter la spécification du package de gestion des clients définie en cours : deux procédures `ajouter_client` surchargées. Gérer toutes les exceptions.

3.1 Spécification du package

```oracle
CREATE OR REPLACE PACKAGE pkg_gestion_clients AS

    -- Procedure 1 : ajout via les attributs individuels
    PROCEDURE ajouter_client(
        p_matr    IN emp.matr%TYPE,
        p_nom     IN emp.nom%TYPE,
        p_sal     IN emp.sal%TYPE,
        p_adresse IN emp.adresse%TYPE,
        p_dep     IN emp.dep%TYPE
    );

    -- Procedure 2 (surcharge) : ajout via un enregistrement %ROWTYPE
    PROCEDURE ajouter_client(
        p_employe IN emp%ROWTYPE
    );

END pkg_gestion_clients;
/
```

Listing 14 – DÃ©claration (spÃ©cification) du package

3.2 Corps du package

```oracle
CREATE OR REPLACE PACKAGE BODY pkg_gestion_clients AS

    -- ---------------------------------------------------------
    -- Procedure 1 : parametres individuels
    -- ---------------------------------------------------------
    PROCEDURE ajouter_client(
        p_matr    IN emp.matr%TYPE,
        p_nom     IN emp.nom%TYPE,
        p_sal     IN emp.sal%TYPE,
        p_adresse IN emp.adresse%TYPE,
        p_dep     IN emp.dep%TYPE
    ) IS
    BEGIN
        INSERT INTO emp (matr, nom, sal, adresse, dep)
        VALUES (p_matr, p_nom, p_sal, p_adresse, p_dep);

        DBMS_OUTPUT.PUT_LINE(
            'Client ' || p_nom || ' ajoute (version individuelle).'
        );
        COMMIT;

    EXCEPTION
        WHEN DUP_VAL_ON_INDEX THEN
            DBMS_OUTPUT.PUT_LINE(
```

```
                'Erreur : le matricule ' || p_matr
                || ' existe deja dans la base.'
            );
            ROLLBACK;

        WHEN VALUE_ERROR THEN
            DBMS_OUTPUT.PUT_LINE(
                'Erreur : valeur invalide pour l un des parametres.'
            );
            ROLLBACK;

        WHEN OTHERS THEN
            DBMS_OUTPUT.PUT_LINE('Erreur inattendue : ' || SQLERRM);
            ROLLBACK;
    END ajouter_client;

    -- -------------------------------------------------------
    -- Procedure 2 (surcharge) : via %ROWTYPE
    -- -------------------------------------------------------
    PROCEDURE ajouter_client(
        p_employe IN emp%ROWTYPE
    ) IS
    BEGIN
        INSERT INTO emp VALUES p_employe;

        DBMS_OUTPUT.PUT_LINE(
            'Client ' || p_employe.nom || ' ajoute (version ROWTYPE).
                '
        );
        COMMIT;

    EXCEPTION
        WHEN DUP_VAL_ON_INDEX THEN
            DBMS_OUTPUT.PUT_LINE(
                'Erreur : le matricule ' || p_employe.matr
                || ' existe deja dans la base.'
            );
            ROLLBACK;

        WHEN VALUE_ERROR THEN
            DBMS_OUTPUT.PUT_LINE(
                'Erreur : valeur invalide dans l enregistrement.'
            );
            ROLLBACK;

        WHEN OTHERS THEN
            DBMS_OUTPUT.PUT_LINE('Erreur inattendue : ' || SQLERRM);
            ROLLBACK;
    END ajouter_client;

END pkg_gestion_clients;
/
```

Listing 15 – Corps (implÃ©mentation) du package

3.3 Tests du package

```
SET SERVEROUTPUT ON;
```

```

-- Test 1 : appel via parametres individuels
BEGIN
    pkg_gestion_clients.ajouter_client(
        p_matr    => 10,
        p_nom     => 'Sophie',
        p_sal     => 3100,
        p_adresse => '15 rue Lecourbe',
        p_dep     => 75000
    );
END;
/

-- Test 2 : appel via ROWTYPE (surcharge)
DECLARE
    v_emp emp%ROWTYPE;
BEGIN
    v_emp.matr    := 11;
    v_emp.nom     := 'Thomas';
    v_emp.sal     := 2900;
    v_emp.adresse := '3 avenue Foch';
    v_emp.dep     := 92000;

    pkg_gestion_clients.ajouter_client(v_emp);
END;
/

-- Test 3 : matricule en doublon
BEGIN
    pkg_gestion_clients.ajouter_client(
        p_matr    => 10,
        p_nom     => 'Doublon',
        p_sal     => 1000,
        p_adresse => 'nulle part',
        p_dep     => 75000
    );
END;
/
```

Listing 16 – Tests : appels normaux et gestion d'exception